אני אתחיל בכך שאני מתנצל על הגשה מאוחרת, שלחתי מייל למירב שישבתי שבעה על סבי והשבעה נגמרת רק ביום ראשון ה 21.12.2025.
אבקש לא להוריד נקודות על הגשה מאוחרת.

מטלה 3 מסמך ניתוח, שחר צרפתי.

Class Diagram

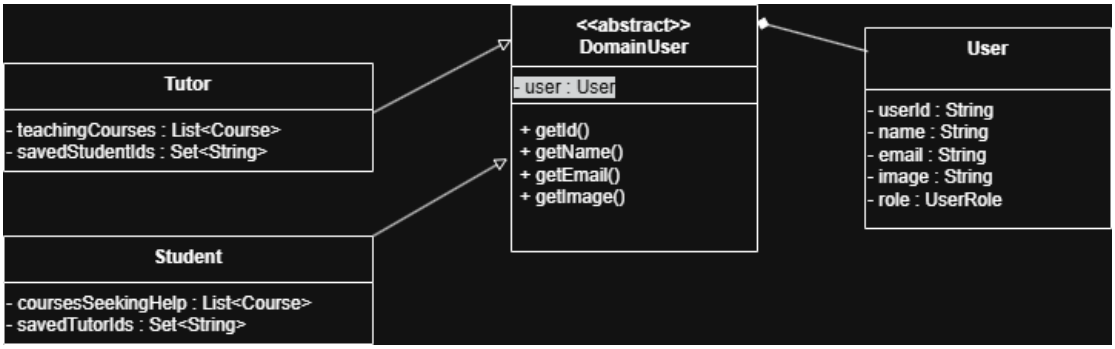
The Class Diagram describes the logical structure of the system and the central classes that compose it. The diagram illustrates the types of objects existing in the system, the fields and methods of each class, and the inheritance and composition relationships between them.

The following classes appear in the diagram:

- **User** – Represents the basic user entity in the system. It includes personal data such as a unique identifier (userId), name, email, image, and role (UserRole).
- **DomainUser** (Abstract) – An abstract class that serves as the foundation for every user within the system's domain. It links general user data to the specific functionality of the system.
- **Student** – Represents a student in the system. It inherits from DomainUser and includes unique lists such as courses for which the student is seeking help (coursesSeekingHelp) and IDs of tutors saved by the student (savedTutorIds).
- **Tutor** – Represents a teacher/instructor in the system. It inherits from DomainUser and includes information about the courses they teach (teachingCourses) and a list of relevant students who have been saved.
- **UserRole** (Enum) – Represents the possible types of roles in the system, such as Student or Tutor.

Relationships between the classes:

- **Inheritance**: The Student and Tutor classes inherit from the abstract class DomainUser, thereby extending the basic functionality of a user within the system.
- **Composition**: The DomainUser class contains an object of type User, which separates the user's identity data from their business logic.
- **Use of Enum**: The User class utilizes UserRole to define the user type and permissions in a strongly typed manner.



Object Diagram

The Object Diagram is a derivative of the Class Diagram and presents a specific snapshot of the system at a given moment. It demonstrates how abstract classes are instantiated into concrete objects with specific data (State) and how the relationships between them are realized.

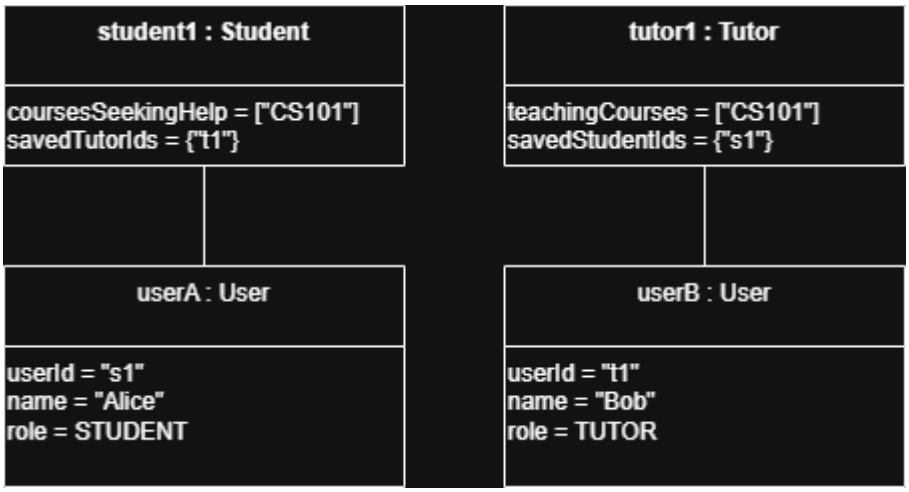
The following instances (Objects) are presented in the diagram:

- **student1** : Student – An instance of the Student class representing the user "Alice." The object shows that the student is seeking help in course "CS101" and has saved the tutor with the ID "t1."
- **userA** : User – Alice's basic data object, containing the unique identifier "s1" and the role (STUDENT).
- **tutor1** : Tutor – An instance of the Tutor class representing the user "Bob." The object shows that Bob teaches the course "CS101" and has saved the student with the ID "s1."
- **userB** : User – Bob's basic data object, containing the unique identifier "t1" and the role (TUTOR).

Significance of the relationships in the diagram:

- **Links** (Object Connections): The lines connecting the instances (for example, between student1 and userA) demonstrate the Composition relationship defined in the Class Diagram.
- **Data Synchronization**: The diagram illustrates how the system maintains consistency – the student's savedTutorIds matches the tutor's userId, indicating an active connection between them in the system.

This diagram complements the structural view and shows how the logical design enables the actual management of connections between learners and tutors.



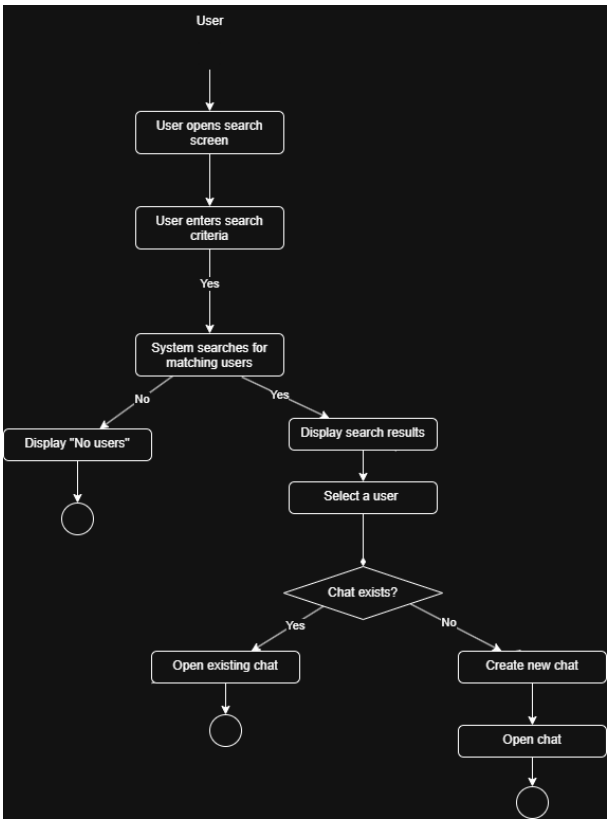
Activity Diagram – User Search and Chat

The Activity Diagram describes the dynamic process of a user searching for a learning partner or tutor and initiating contact. This process demonstrates the system's business logic beyond its static structure.

The process stages presented in the diagram:

- **Initiating Search:** The user opens the search screen and enters specific search criteria (e.g., course name or user type).
- **System Search:** The system performs a search for matching users.
    - If no results are found, the system displays a "No users" message, and the process ends.
    - If matches are found, the search results are displayed to the user.
- **User Selection:** The user selects a specific person from the results list to contact.
- **Chat Management:** The system checks whether a chat history already exists between the two users:
    - If a chat exists, the system opens the existing conversation.
    - If no chat exists, the system creates a new chat session and opens it.

This diagram illustrates how the system manages information flow and logical decision-making to facilitate efficient communication between users.



Sequence Diagram Description – Messaging Process

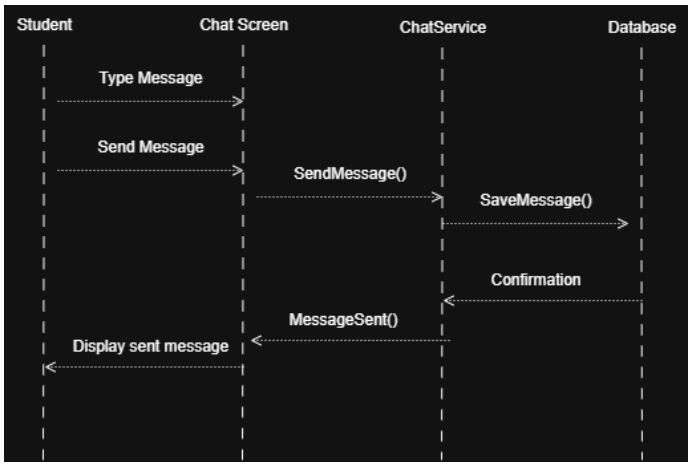
The Sequence Diagram models the step-by-step interaction between the Student, the Chat Screen, the ChatService, and the Database during a live messaging event. This process is critical for the real-time communication functionality of the system.

The following participants are involved in the process:

- **Student**: The human actor initiating the communication.
- **Chat Screen**: The user interface (UI) layer that captures user input and displays system responses.
- **ChatService**: The logic layer responsible for processing the message and coordinating with the storage layer.
- **Database**: The persistence layer where messages are securely stored.

The sequence of interactions is as follows:

1. **Input and Initiation**: The Student types a message and triggers the "Send" action on the Chat Screen.
2. **Service Request**: The Chat Screen calls the SendMessage() method on the ChatService.
3. **Data Persistence**: The ChatService invokes SaveMessage() to store the content in the Database.
4. **Confirmation Flow**:
    - The Database sends a **Confirmation** back to the ChatService once the message is saved.
    - The ChatService notifies the UI via the MessageSent() callback.
5. **User Feedback**: Finally, the Chat Screen updates the display for the Student to show that the message has been successfully sent.

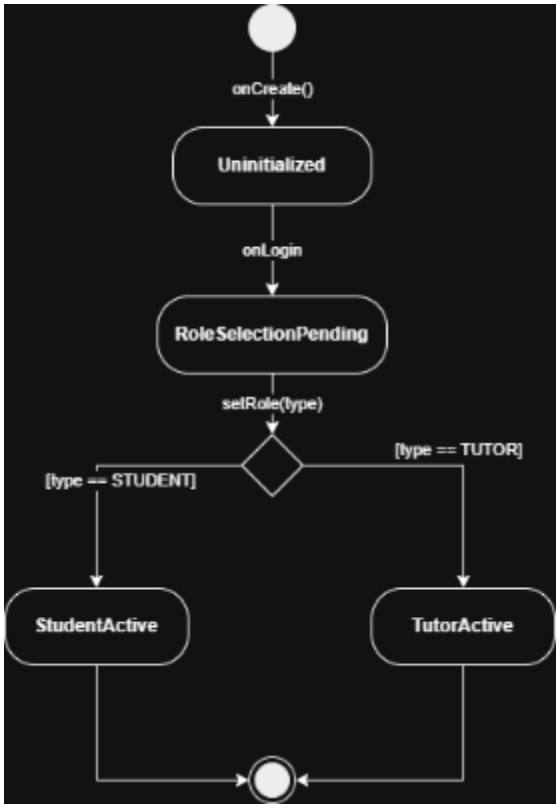


State Machine Diagram Description – User Role Lifecycle

The State Machine Diagram illustrates the various states a user object transitions through during its initialization phase. This is a critical process where a generic profile becomes a specialized entity (either a Student or a Tutor).

The states and transitions presented include:

- **Initial State**: Represented by the top solid circle, this indicates the creation of a new user profile in the system.
- **Uninitialized State**: After the onCreate() event, the user exists as a basic record without any specific domain data or role.
- **RoleSelectionPending State**: Triggered by the onLogin event, the system moves the user to a state where they are required to choose their function in the system.
- **Choice Point (Diamond)**: This UML symbol represents the logic branch for the setRole(type) event.
- **Active States (StudentActive / TutorActive)**: Based on the guard conditions [type == STUDENT] or [type == TUTOR], the object settles into its final operational state.
- **Final State**: The bullseye symbol at the bottom represents the completion of the initialization lifecycle, meaning the object is now fully functional.

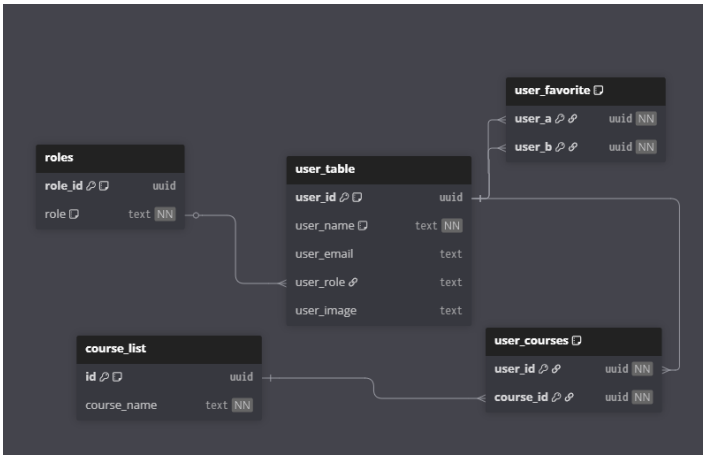


Entity-Relationship Diagram (ERD) Description

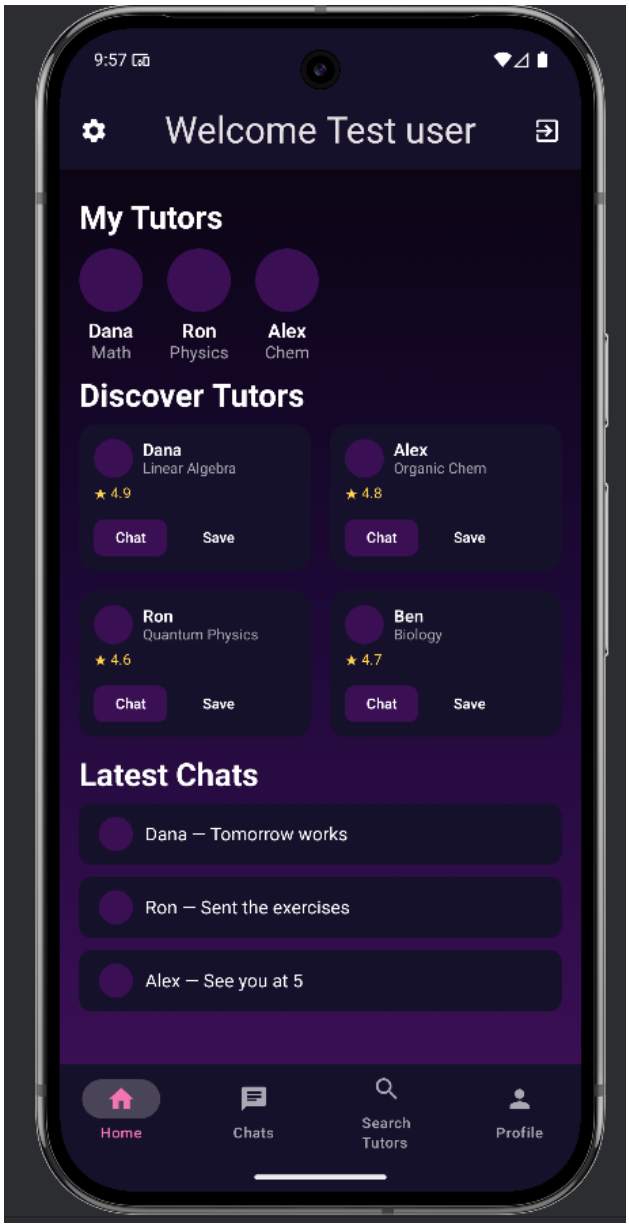
The ERD provides a complete data model for the system, ensuring data persistence and structural integrity. The model is normalized to **Third Normal Form (3NF)** to eliminate data redundancy and transitive dependencies.

Entities and Schema Structure:

- **user_table**: The central entity storing profile information. It includes user_id as the Primary Key (PK) and a Foreign Key (FK) to the roles table.
- **roles**: A lookup table used to normalize user roles. By separating roles into a distinct table with a unique constraint, the system avoids repetitive string data in the main user table.
- **course_list**: A master list of all available subjects in the system.
- **user_courses**: A bridge (junction) table that resolves the many-to-many relationship between users and courses. This ensures that each record is atomic, fulfilling 1NF and 3NF requirements.
- **user_favorite**: A self-referencing junction table that handles the "Favorite" logic (e.g., a Student saving a Tutor). It links two distinct user_id entries from the user_table.



System Screens Structure



Home Screen: Acts as the central hub of the application, displaying the user's saved tutors ("My Tutors"), personalized recommendations ("Discover Tutors"), and ongoing conversations ("Latest Chats").

Settings (Gear Icon): Located at the top left, this allows users to access application configurations, account preferences, and system settings.

Navigation Bar: Positioned at the bottom, it provides quick access to the main sections of the app: Home, Chats, Search Tutors, and Profile.

Logout (Exit Icon): Found at the top right, this button securely ends the current session and returns the user to the login screen.

Profile (User Icon): Accessible via the rightmost tab in the navigation bar, this section allows users to view and edit their personal information, image, and role.

Design Patterns (MVVM)

In our project, we make use of the familiar **MVVM (Model-View-ViewModel)** design pattern to manage responsibility and concern separation. This pattern suits us best as it is commonly practiced with Android views databinding and creates live, reactive views linked to a remote database without responsibility mixing.

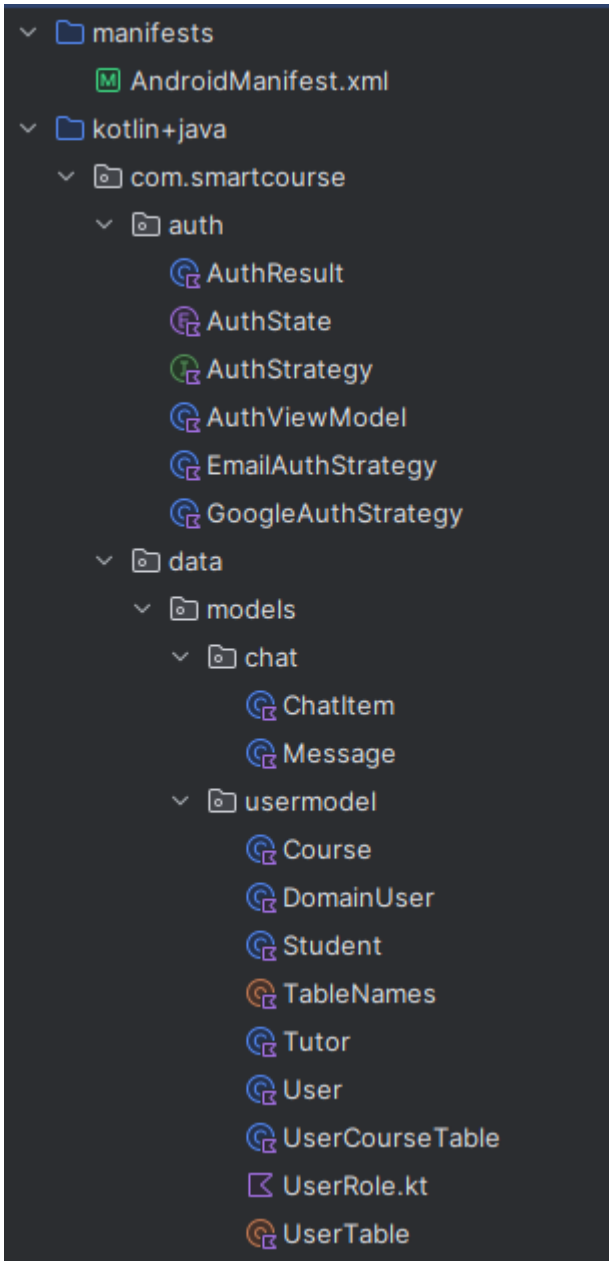
- **View**: The Android screens, such as the **Home** and **Chat** pages, serve as the reactive UI layer that captures user interactions like "Search" or "Send Message".
- **ViewModel**: This layer manages the application logic and state, such as processing **Role Selection** or coordinating the **SendMessage()** flow between the UI and the backend.
- **Model**: The data layer is comprised of a **3NF normalized database** and domain classes that handle persistent storage for users, courses, and messages.
- **Separation of Concerns**: This structure ensures that UI updates are driven by data changes (databinding) without mixing the interface code with the database logic.

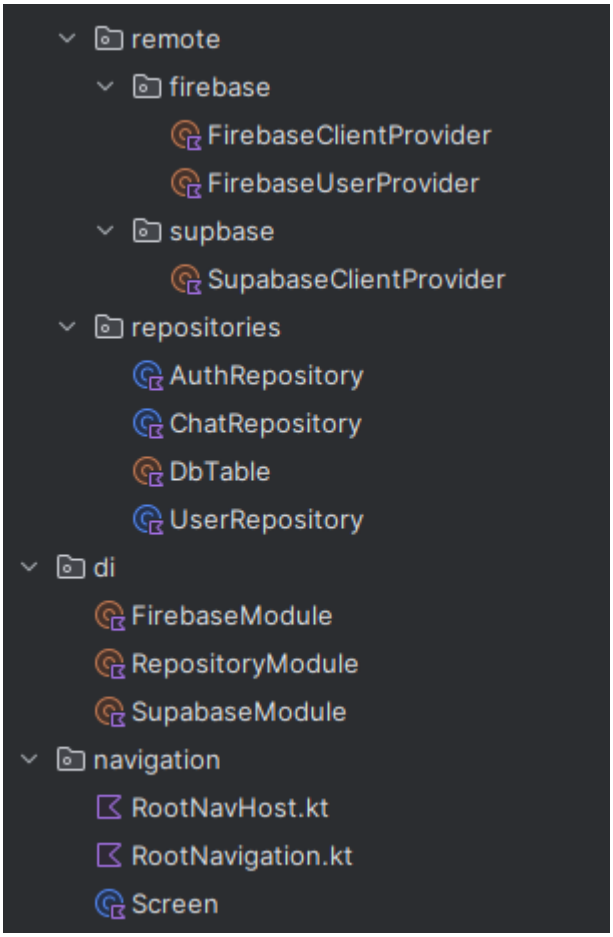
Project Structure & Architecture

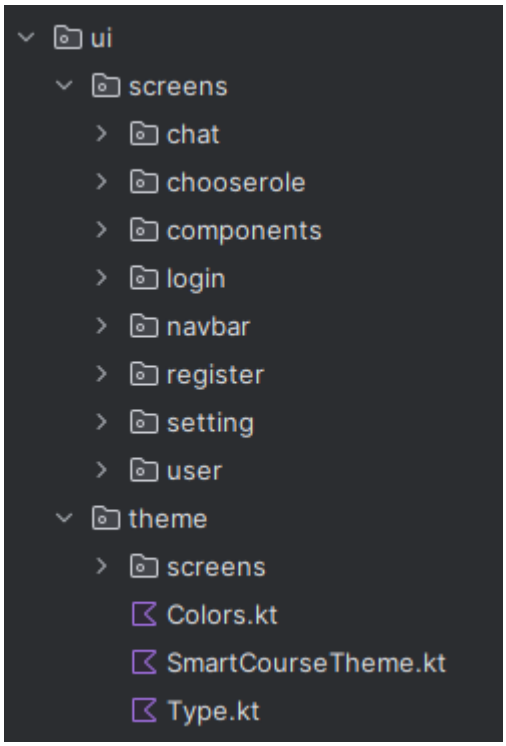
As can be seen in our project hierarchy, we structure our project strictly around the **MVVM (Model-View-ViewModel)** design pattern to ensure a clean separation of concerns and a reactive user experience.

The Architecture Layers:

- **Views (/ui/screens)**: The UI is organized into feature-based packages such as chat, chooserole, login, and user. These screens act as the reactive layer, capturing user inputs and observing data changes to update the interface dynamically.
- **ViewModels (/auth, /data)**: We utilize various ViewModels, such as the AuthViewModel, to relay actions performed in the views to the underlying data layer. These components manage the application state, such as the transition from Uninitialized to an active role like StudentActive.
- **Models and Data (/data/models, /data/repositories)**: The data layer acts as the single source of truth, consisting of repositories that hook into the **Firebase** and **Supabase** databases. The models, including Student, Tutor, and Course, are structured according to a normalized **3NF data model** to ensure structural integrity and prevent redundancy.
- **Dependency Injection & Navigation (/di, /navigation)**: To further maintain a decoupled structure, we use a dedicated **DI module** to provide database clients and repositories, while a **Root Navigation** system manages the flow between screens based on the user's current state.


manifests
AndroidManifest.xml
kotlin+java
com.smartcourse
auth
AuthResult
AuthState
AuthStrategy
AuthViewModel
EmailAuthStrategy
GoogleAuthStrategy
data
models
chat
ChatItem
Message
usermodel
Course
DomainUser
Student
TableNames
Tutor
User
UserCourseTable
UserRole.kt
UserTable


remote
firebase
FirebaseClientProvider
FirebaseUserProvider
supbase
SupabaseClientProvider
repositories
AuthRepository
ChatRepository
DbTable
UserRepository
di
FirebaseModule
RepositoryModule
SupabaseModule
navigation
RootNavHost.kt
RootNavigation.kt
Screen


ui
screens
chat
chooserole
components
login
navbar
register
setting
user
theme
screens
Colors.kt
SmartCourseTheme.kt
Type.kt